

mxSDK 开发文档

一、关于sdk的依赖说明

注意: mxSdk 已内置以下依赖库, 无需额外添加:

- **OpenCV 4.13.0**: 图像处理库
- **serialport**: 串口通信库

二、常用API列表

1	share()	用于获取ConnectManager对象
2	init(@NonNull Application _application)	用于初始化ConnectManager类中，必须调用
3	destroy()	释放ConnectManager所有资源，再次使用调用必须重新初始化init方法
4	isEnabled()	判断蓝牙是否打开
5	enable()	打开蓝牙
6	disable()	关闭蓝牙
7	public void discoverBluetoothDevice(float scanTime)	扫描蓝牙设备
8	public void cancelDiscoveryBluetoothDevice()	停止蓝牙设备扫描
9	isDiscoveringBluetoothDevice()	判断是否正在扫描
10	public synchronized void discoverDistNetDevice()	扫描配网设备
11	public synchronized void cancelDiscoverDistNetDevice()	停止配网设备扫描
12	public synchronized void distributionNetwork(DistNetDevice distNetDevice, String ssid, String password, float timeoutValue)	对配网设备进行配网
13	public void discoverWifiDevice(float scanTime)	扫描WiFi设备
14	public void cancelDiscoverWifiDevice()	停止WiFi设备扫描
15	public List getSerialDevices()	获取串口设备
16	public void connect(Device device)	连接设备
17	public void disconnect()	断开设备连接
18	public void setWithSendMultiRowDataPacket(MultiRowData multiRowData)	按默认协议设置并发送打印数据

1	share()	用于获取ConnectManager对象
19	public void setWithSendMultiRowDataPacket(MultiRowData multiRowData, int fh)	按照指定的协议设置并发送打印 数据
20	public void setWithSendOtaPacket(byte[] data)	设置并发送ota数据
21	public void setWithSendOtaPacket(byte[] data, int fn)	按照指定的协议设置并发送ota 数据
22	public void setWithSendLogoPacket(LogoData logoData)	按照默认协议设置并发送打印机 默认打印内容
23	public void setWithSendLogoPacket(LogoData logoData, int fn)	按照指定协议设置并发送打印机 默认打印内容
24	public Boolean isConnected()	判断打印机是否连接
25	public Boolean isConnectedDevice(Device device)	判断某个设备是否连接
26	public Boolean isConnectingDevice(Device device)	判断某个设备是否正在连接
27	public Device getConnectedDevice()	获取已连接设备
28	public void readPrinterHeadParameters()	读取打印头参数
29	public void readPrinterHeadParameters(int delayTime)	延时delayTime毫秒读取打印头参 数（事件单位毫秒）
30	public void writePrinterHeadParameters(int printer_head, int pix)	设置打印头及打印分辨率（打印 头分1、2、3、4个序号）
31	public void writePrinterHeadParameters(int printer_head, int pix, int delayTime)	延时delayTime毫秒设置打印头及 打印分辨率（打印头分1、2、3、 4个序号，时间单位毫秒）
32	public void writePrinterHeadParameters(int printer_head, int l_pix, int p_pix, int distance)	设置打印头及打印分辨率及延迟 打印距离（打印头分1、2、3、4 个序号，时间单位毫秒）

1	share()	用于获取ConnectManager对象
33	public void writePrinterHeadParameters(int printer_head, int l_pix, int p_pix, int distance, int delayTime)	延时delayTime毫秒设置打印头及打印分辨率及延迟打印距离（打印头分1、2、3、4个序号，时间单位毫秒）
34	public void readCirculationAndRepeatTime()	读取打印机的打印循环次数和重复次数
35	public void readCirculationAndRepeatTime(int delayTime)	延时delayTime毫秒读取打印机的打印循环次数和重复次数
36	public void writeCirculationAndRepeatTime(int circulation_time, int repeat_time)	设置打印机循环打印次数和重复打印次数
37	public void writeCirculationAndRepeatTime(int circulation_time, int repeat_time, int delayTime)	延时delayTime毫秒重复设置打印机循环打印次数和重复打印次数
38	public void readPrintDirection()	读取打印方向
39	public void readPrintDirection(int delayTime)	延时delayTime毫秒读取打印方向
40	public void writePrintDirection(int horizontalDirection, int verticalDirection)	设置水平和垂直打印方向
41	public void writePrintDirection(int horizontalDirection, int verticalDirection, int delayTime)	延时delayTime毫秒设置水平和垂直打印方向
42	public void readSoftwareInfo()	读取软件信息
43	public void readSoftwareInfo(int delayTime)	延时delayTime毫秒读取软件信息
44	public void readBattery()	读取电量信息
45	public void readBattery(int delayTime)	延时delayTime毫秒读取电量信息
46	public static void Logolmage2Data(Context context, Logolmage logolmage, int threshold, boolean transparentToWhiteAuto, OnCreateLogoDataListener onCreateLogoDataListener)	生成默认打印内容数据LogoData的方法，该方法内部执行线程，并以事件的形式返回生成结果

1	share()	用于获取ConnectManager对象
47	public static LogoData LogoImage2Data(Context context, LogoImage logoImage, int threshold, boolean transparentToWhiteAuto)	生成打印机默认打印数据 LogoData的方法，图片处理过程为耗时任务，需要自己在外边创建线程来配合使用
48	public static void bitmap2MultiRowData(Context context, MultiRowImage multiRowImage, int threshold, boolean clearBackground, boolean dithering, boolean compress, boolean flipHorizontally, boolean transparentToWhiteAuto, boolean thumbToSimulation, OnCreateMultiRowDataListener onCreateMultiRowDataListener)	生成打印数据MultiRowData，该方法内部自己维护线程，并以事件的方式返回执行结果
49	public static MultiRowData bitmap2MultiRowData(Context context, MultiRowImage multiRowImage, int threshold, boolean clearBackground, boolean dithering, boolean compress, boolean flipHorizontally, boolean transparentToWhiteAuto, boolean thumbToSimulation)	生成打印数据MultiRowData，图片处理过程为耗时任务，需要在使用时创建线程来执行
50	public static void image2MultiRowImage(Context context, Uri uri, RowLayoutDirection rowLayoutDirection, OnCreateMultiRowImageListener onCreateMultiRowImageListener)	将一张普通的图片转成MultiRowImage，参数context 上下文对象，uri图片Uri，rowLayoutDirection裁剪方向，onCreateMultiRowImageListener用于将处理的结果以事件的形式传递出来
51	public static void image2MultiRowImage(Context context, Uri uri, RowLayoutDirection rowLayoutDirection, int ignoreLastRowIfHeightLess, OnCreateMultiRowImageListener onCreateMultiRowImageListener)	将一张普通的图片转成MultiRowImage，参数context 上下文对象，uri图片Uri，rowLayoutDirection裁剪方向，ignoreLastRowIfHeightLess图片最后一拼忽略值，用于处理最后一拼尺寸太小打印意义不大的情况下进行忽略这一拼，onCreateMultiRowImageListener

1	share()	用于获取 ConnectManager 对象
		用于将处理的结果以事件的形式传递出来
52	<pre>public static void image2MultiRowImage(Context context, String imagePath, RowLayoutDirection rowLayoutDirection, OnCreateMultiRowImageListener onCreateMultiRowImageListener)</pre>	将一张普通的图片转成 MultiRowImage ，参数context 上下文对象，imagePath图片的存储路径，rowLayoutDirection裁剪方向， onCreateMultiRowImageListener 用于将处理的结果以事件的形式传递出来
53	<pre>public static void image2MultiRowImage(Context context, String imagePath, RowLayoutDirection rowLayoutDirection, int ignoreLastRowIfHeightLess, OnCreateMultiRowImageListener onCreateMultiRowImageListener)</pre>	将一张普通的图片转成 MultiRowImage ，参数context 上下文对象，imagePath图片的存储路径，rowLayoutDirection裁剪方向， ignoreLastRowIfHeightLess 图片最后一拼忽略值，用于处理最后一拼尺寸太小打印意义不大的情况下进行忽略这一拼， onCreateMultiRowImageListener 用于将处理的结果以事件的形式传递出来
54	<pre>public static void image2MultiRowImage(Context context, Bitmap bitmap, RowLayoutDirection rowLayoutDirection, OnCreateMultiRowImageListener onCreateMultiRowImageListener)</pre>	将一张普通的图片转成 MultiRowImage ，参数context 上下文对象，bitmap要裁剪的图片，rowLayoutDirection裁剪方向， onCreateMultiRowImageListener 用于将处理的结果以事件的形式传递出来
55	<pre>public static void image2MultiRowImage(Context context, Bitmap bitmap, RowLayoutDirection rowLayoutDirection, int ignoreLastRowIfHeightLess, OnCreateMultiRowImageListener onCreateMultiRowImageListener)</pre>	将一张普通的图片转成 MultiRowImage ，参数context 上下文对象，bitmap要裁剪的图片，rowLayoutDirection裁剪方向， ignoreLastRowIfHeightLess 图片最后一拼忽略值，用于处理

1	<code>share()</code>	用于获取ConnectManager对象
		最后一拼尺寸太小打印意义不大的情况下进行忽略这一拼， <code>onCreateMultiRowImageListener</code> 用于将处理的结果以事件的形式传递出来
56	<code>public void startPrint()</code>	发送打印指令，和打印机上打印按钮功能相同

三、API使用及示例

1、share()获取ConnectManager单例

```
ConnectManager connectManager = ConnectManager.share();
```

2、init(@NonNull Application _application)初始化ConnectManager方法

```
ConnectManager.share().init(this);
```

3、destroy()资源释放

```
ConnectManager.share().destroy();
```

4、isEnabled()判断蓝牙是否打开

```
ConnectManager.share().isEnabled();
```

5、enable()打开蓝牙

```
ConnectManager.share().enable();  
//也可以使用intent来打开  
Intent intent = new Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE);  
activity.startActivityForResult(intent, REQUEST_OPEN_BT_CODE);
```

6、disable() 关闭蓝牙

```
ConnectManager.share().disable();
```

7、discoverBluetoothDevice(float scanTime)扫描蓝牙设备

```
//发现设备事件的注册
ConnectManager.share().registerDeviceDiscoverListener(onDeviceDiscoverListener);
//发现设备事件的注销
ConnectManager.share().unregisterDeviceDiscoverListener(onDeviceDiscoverListener);

//参数scanTime 扫描时长, 单位为秒
ConnectManager.share().discoverBluetoothDevice(20);
//发现设备事件
private final ConnectManager.OnDeviceDiscoverListener onDeviceDiscoverListener = new
ConnectManager.OnDeviceDiscoverListener() {
    @Override
    public void onDeviceStartDiscover() {
        RBQLog.i("开始扫描设备");
    }

    @Override
    public void onDeviceStopDiscover() {
        RBQLog.i("停止扫描蓝牙设备");
    }

    @Override
    public void onDeviceDiscovered(Device device) {

    }

};
```

8、cancelDiscoveryBluetoothDevice()取消扫描蓝牙设备

```
ConnectManager.share().cancelDiscoveryBluetoothDevice();
```

9、isDiscoveringBluetoothDevice()判断设备是否正在扫描蓝牙设备

```
ConnectManager.share().isDiscoveringBluetoothDevice();
```

10、discoverDistNetDevice()搜索配网设备


```

//注册搜索设备事件
ConnectManager.share().registerDistNetDeviceDiscoverListener(onDistNetDeviceDiscoverLis
tener);
//注销配网设备事件
ConnectManager.share().unregisterDistNetDeviceDiscoverListener(onDistNetDeviceDiscoverL
istener);

//搜索配网设备
ConnectManager.share().discoverDistNetDevice();
//事件示例
ConnectManager.OnDistNetDeviceDiscoverListener onDistNetDeviceDiscoverListener = new
ConnectManager.OnDistNetDeviceDiscoverListener() {
    @Override
    public void onDistNetDeviceDiscoverStart() {
        //开始搜索配网设备
    }

    @Override
    public void onDistNetDeviceDiscover(DistNetDevice device) {
        //搜索到配网设备
    }

    @Override
    public void onDistNetDeviceDiscoverCancel() {
        //取消配网设备搜索
    }
};

```

11、cancelDiscoverDistNetDevice()取消配网设备搜索

```

//停止配网设备搜索
ConnectManager.share().cancelDiscoverDistNetDevice();

```

12、distributionNetwork(DistNetDevice distNetDevice, String ssid, String password, float timeoutValue)对wifi设备进行配网

```

//事件注册
ConnectManager.share().registerDistributionNetworkListener(onDistributionNetworkListene
r);
//事件注销
ConnectManager.share().unregisterDistributionNetworkListener(onDistributionNetworkListe
ner);

//启动配网 参数分别为 配网设备:DistNetDevice WiFi账号:SSID 密码:PASSWORD 超时时间:
timeoutValue 单位s
ConnectManager.share().distributionNetwork(distNetDevice, SSID, PASSWORD, 100);

//事件
ConnectManager.OnDistributionNetworkListener onDistributionNetworkListener = new
ConnectManager.OnDistributionNetworkListener() {
    @Override
    public void onDistributionNetworkStart() {
        //配网开始
    }

    @Override
    public void onDistributionNetworkSucceed(Device device) {
        //配网成功
    }

    @Override
    public void onDistributionNetworkFail() {
        //配网失败
    }

    @Override
    public void onDistributionNetworkTimeOut() {
        //配网超时
    }
};

```

13、discoverWifiDevice(float scanTime)扫描WiFi设备

```

//事件注册
ConnectManager.share().registerDeviceDiscoverListener(onDeviceDiscoverListener);
//事件注销
ConnectManager.share().unregisterDeviceDiscoverListener(onDeviceDiscoverListener);

//搜索已配好网的WiFi设备
ConnectManager.share().discoverWifiDevice();
//事件
private final ConnectManager.OnDeviceDiscoverListener onDeviceDiscoverListener = new
ConnectManager.OnDeviceDiscoverListener() {
    @Override
    public void onDeviceStartDiscover() {
        RBQLog.i("开始扫描设备");
    }

    @Override
    public void onDeviceStopDiscover() {
        RBQLog.i("停止扫描蓝牙设备");
    }
    @Override
    public void onDeviceDiscovered(Device device) {
        RBQLog.i("发现设备");
    }
};

```

14、cancelDiscoverWifiDevice() 取消WiFi设备扫描

```

//取消WiFi设备搜索
ConnectManager.share().cancelDiscoverWifiDevice();

```

15、getSerialDevices() 获取串口设备

```

// 获取串口设备
List<Device> devices = ConnectManager.share().getSerialDevices();

```

16、connect(Device device) 连接设备

```

//连接设备事件注册
ConnectManager.share().registerDeviceConnectListener(onDeviceConnectListener);
//连接设备事件注销
ConnectManager.share().unregisterDeviceConnectListener(onDeviceConnectListener);

//设备连接
ConnectManager.share().connect(device);

//连接设备事件示例
ConnectManager.OnDeviceConnectListener onDeviceConnectListener = new
ConnectManager.OnDeviceConnectListener() {
    @Override
    public void onDeviceConnectStart(Device device) {

    }

    @Override
    public void onDeviceConnectSucceed(Device device) {

    }

    @Override
    public void onDeviceDisconnect(Device device) {

    }

    @Override
    public void onDeviceConnectFail(Device device, String error) {

    }

};

```

17、disconnect()断开设备连接

```

ConnectManager.share().disconnect();

```

18、public void setWithSendMultiRowDataPacket(MultiRowData multiRowData)按照默认的协议发送打印数据

备注：MultiRowData 关于MultiRowData对象在会在后边的篇幅中详细讲解，它用来代表多张图片或者一张图裁剪成多张连续的拼接图生成的数据

```

//注册数据传送事件
ConnectManager.share().registerDataProgressListener(onDataProgressListener);
//反注册数据传输事件
ConnectManager.share().unregisterDataProgressListener(onDataProgressListener);

//设置并发送数据
ConnectManager.share().setWithSendMultiRowDataPacket(multiRowData);
// 数据传输事件 注意：这里的时间均为时间戳
ConnectManager.OnDataProgressListener onDataProgressListener = new
ConnectManager.OnDataProgressListener() {
    @Override
    public void onDataProgressStart(float size, int progress, long startTime) {

    }

    @Override
    public void onDataProgress(float size, int progress, long startTime, long
currentTime) {

    }

    @Override
    public void onDataProgressFinish(float size, long startTime, long currentTime)
{

    }

    @Override
    public void onDataProgressError(String error, int code) {

    }

};

```

19、public void setWithSendMultiRowDataPacket(MultiRowData multiRowData, int fh) 按照指定的协议发送打印数据

注意：sdk内支持多重协议，打印机具体支持那种协议，请根据当前所开发的打印机支持的实际协议来选择协议

```

//注册数据传送事件
ConnectManager.share().registerDataProgressListener(onDataProgressListener);
//反注册数据传输事件
ConnectManager.share().unregisterDataProgressListener(onDataProgressListener);

//设置并发送数据
ConnectManager.share().setWithSendMultiRowDataPacket(multiRowData,STX_A);
// 数据传输事件 注意：这里的时间均为时间戳
ConnectManager.OnDataProgressListener onDataProgressListener = new
ConnectManager.OnDataProgressListener() {
    @Override
    public void onDataProgressStart(float size, int progress, long startTime) {

    }

    @Override
    public void onDataProgress(float size, int progress, long startTime, long
currentTime) {

    }

    @Override
    public void onDataProgressFinish(float size, long startTime, long currentTime)
{

    }

    @Override
    public void onDataProgressError(String error, int code) {

    }

};

```

20、 public void setWithSendOtaPacket(byte[] data) 使用默认的协议发送ota数据进行ota升级

```

//注册数据传送事件
ConnectManager.share().registerDataProgressListener(onDataProgressListener);
//反注册数据传输事件
ConnectManager.share().unregisterDataProgressListener(onDataProgressListener);

//使用默认协议发送ota数据
ConnectManager.share().setWithSendOtaPacket(buffer);

// 数据传输事件 注意：这里的时间均为时间戳
ConnectManager.OnDataProgressListener onDataProgressListener = new
ConnectManager.OnDataProgressListener() {
    @Override
    public void onDataProgressStart(float size, int progress, long startTime) {

    }

    @Override
    public void onDataProgress(float size, int progress, long startTime, long
currentTime) {

    }

    @Override
    public void onDataProgressFinish(float size, long startTime, long currentTime)
{

    }

    @Override
    public void onDataProgressError(String error, int code) {

    }

};

```

21、public void setWithSendOtaPacket(byte[] data, int fn) 按照指定的协议发送ota数据进行ota升级

```

//注册数据传送事件
ConnectManager.share().registerDataProgressListener(onDataProgressListener);
//反注册数据传输事件
ConnectManager.share().unregisterDataProgressListener(onDataProgressListener);

//使用STX_A协议发送ota数据
ConnectManager.share().setWithSendOtaPacket(buffer,STX_A);

// 数据传输事件 注意：这里的时间均为时间戳
ConnectManager.OnDataProgressListener onDataProgressListener = new
ConnectManager.OnDataProgressListener() {
    @Override
    public void onDataProgressStart(float size, int progress, long startTime) {

    }

    @Override
    public void onDataProgress(float size, int progress, long startTime, long
currentTime) {

    }

    @Override
    public void onDataProgressFinish(float size, long startTime, long currentTime)
{

    }

    @Override
    public void onDataProgressError(String error, int code) {

    }

};

```

22、public void setWithSendLogoPacket(LogoData logoData) 使用默认协议修改默认打印内容


```

//注册数据传送事件
ConnectManager.share().registerDataProgressListener(onDataProgressListener);
//反注册数据传输事件
ConnectManager.share().unregisterDataProgressListener(onDataProgressListener);

//使用默认协议发送打印机默认打印内容 图片尺寸要求(2000x552) 在尺寸不足时sdk内部会去自动调整
ConnectManager.share().setWithSendLogoPacket(logoData);

// 数据传输事件 注意：这里的时间均为时间戳
ConnectManager.OnDataProgressListener onDataProgressListener = new
ConnectManager.OnDataProgressListener() {
    @Override
    public void onDataProgressStart(float size, int progress, long startTime) {

    }

    @Override
    public void onDataProgress(float size, int progress, long startTime, long
currentTime) {

    }

    @Override
    public void onDataProgressFinish(float size, long startTime, long currentTime)
{

    }

    @Override
    public void onDataProgressError(String error, int code) {

    }

};

```

23、public void setWithSendLogoPacket(LogoData logoData, int fn)使用指定协议修改默认打印内容

```

//注册数据传送事件
ConnectManager.share().registerDataProgressListener(onDataProgressListener);
//反注册数据传输事件
ConnectManager.share().unregisterDataProgressListener(onDataProgressListener);

//使用STX_A协议发送打印机默认打印内容 图片尺寸要求(2000x552) 在尺寸不足时sdk内部会去自动调整
ConnectManager.share().setWithSendLogoPacket(logoData, STX_A);

// 数据传输事件 注意：这里的时间均为时间戳
ConnectManager.OnDataProgressListener onDataProgressListener = new
ConnectManager.OnDataProgressListener() {
    @Override
    public void onDataProgressStart(float size, int progress, long startTime) {

    }

    @Override
    public void onDataProgress(float size, int progress, long startTime, long
currentTime) {

    }

    @Override
    public void onDataProgressFinish(float size, long startTime, long currentTime)
{

    }

    @Override
    public void onDataProgressError(String error, int code) {

    }

};

```

24、public Boolean isConnected() 判断打印机是否连接

```
ConnectManager.share().isConnected();
```

25、public Boolean isConnectedDevice(Device device) 判断某个打印是否已连接

```
ConnectManager.share().isConnectedDevice(device)
```

26、public Boolean isConnectingDevice(Device device)判断该设备是否正在连接

```
ConnectManager.share().isConnectingDevice(device);
```

27、**public Device getConnectedDevice()**获取已连接的设备，如果没有设备连接则，返回null

```
Device connectedDevice = ConnectManager.share().getConnectedDevice();
```

28、**public void readPrinterHeadParameters()**读取打印头参数

```
//注册打印头读取事件
ConnectManager.share().registerReceiveMessageListener(onReceiveMsgListener);
//注销
ConnectManager.share().unregisterReceiveMessageListener(onReceiveMsgListener);

// 读取打印头参数
ConnectManager.share().readPrinterHeadParameters();

// 事件
ConnectManager.OnReceiveMsgListener onReceiveMsgListener = new
ConnectManager.OnReceiveMsgListener() {
    @Override
    public void onReadPrinterHeadParameter(Device device, int headValue, int l_pix, int
p_pix, int distance) {
        // 返回打印头参数数据
    }

    @Override
    public void onReadCirculationAndRepeatTime(Device device, int circulation_time, int
repeat_time) {
        // 返回循环次数和重复次数数据
    }

    @Override
    public void onReadDirection(Device device, int oldHorizontalDirection, int
horizontalDirection, int oldVerticalDirection, int verticalDirection) {
        //返回打印方向数据 0和1
    }

    @Override
    public void onReadSoftwareInfo(Device device, String id, String name, String
mcu_version, String mcu_date) {
        //返回软件版本
    }

    @Override
    public void onReadTemperature(Device device, int temp) {

    }

    @Override
    public void onReadBattery(Device device, int bat) {
        //读取电量
    }
}
```

```

    }

    @Override
    public void onReadCartridgeId(Device device, String cartridgeId) {
        //读取墨盒id
    }

    @Override
    public void onReadSilentState(Device device, boolean silentState) {
        //读取静音状态
    }

    @Override
    public void onReadAutoPowerOffState(Device device, boolean autoPowerOff) {
        //读取自动关机状态
    }

    @Override
    public void onReadContinuousPrintState(Device device, boolean continuousPrintState)
{
        //读取连续打印状态
    }

    @Override
    public void onError(Device device, String error) {

    }

};

```

29、public void readPrinterHeadParameters(int delayTime)按照指定的延时时间，读取打印头参数（事件单位毫秒）

```
//注册打印头读取事件
ConnectManager.share().registerReceiveMessageListener(onReceiveMsgListener);
//注销
ConnectManager.share().unregisterReceiveMessageListener(onReceiveMsgListener);

// 读取打印头参数 delayTime毫秒后自动发出指令并进行读取
ConnectManager.share().readPrinterHeadParameters(delayTime);

// 事件
ConnectManager.OnReceiveMsgListener onReceiveMsgListener = new
ConnectManager.OnReceiveMsgListener() {
    @Override
    public void onReadPrinterHeadParameter(Device device, int headValue, int l_pix, int
p_pix, int distance) {
        // 返回打印头参数数据
    }

    @Override
    public void onReadCirculationAndRepeatTime(Device device, int circulation_time, int
repeat_time) {
        // 返回循环次数和重复次数数据
    }

    @Override
    public void onReadDirection(Device device, int oldHorizontalDirection, int
horizontalDirection, int oldVerticalDirection, int verticalDirection) {
        //返回打印方向数据 0和1
    }

    @Override
    public void onReadSoftwareInfo(Device device, String id, String name, String
mcu_version, String mcu_date) {
        //返回软件版本
    }

    @Override
    public void onReadTemperature(Device device, int temp) {

    }

    @Override
    public void onReadBattery(Device device, int bat) {
        //读取电量
    }
}
```

```

    }

    @Override
    public void onReadCartridgeId(Device device, String cartridgeId) {
        //读取墨盒id
    }

    @Override
    public void onReadSilentState(Device device, boolean silentState) {
        //读取静音状态
    }

    @Override
    public void onReadAutoPowerOffState(Device device, boolean autoPowerOff) {
        //读取自动关机状态
    }

    @Override
    public void onReadContinuousPrintState(Device device, boolean continuousPrintState)
{
        //读取连续打印状态
    }

    @Override
    public void onError(Device device, String error) {

    }

};

```

30、public void writePrinterHeadParameters(int printer_head, int pix)设置打印头及打印分辨率（打印头分1、2、3、4个序号，分辨率支持 600x600 1200x1200）

```

/**
 * printer_head 打印头编号
 * pix 分辨率
 */
ConnectManager.share().writePrinterHeadParameters(int printer_head, int pix);
// 事件注册和监听 同 readPrinterHeadParameters()

```

31、public void writePrinterHeadParameters(int printer_head, int pix, int delayTime) 延时delayTime毫秒设置打印头及打印分辨率（打印头分1、2、3、4个序号，时间单位毫秒）

```

/**
 * printer_head 打印头编号
 * pix 分辨率
 */
ConnectManager.share().writePrinterHeadParameters(int printer_head, int pix, int
delayTime);
// 事件注册和监听 同 readPrinterHeadParameters()

```

32、public void writePrinterHeadParameters(int printer_head, int l_pix, int p_pix, int distance)设置打印头及打印分辨率及延迟打印距离（打印头分1、2、3、4个序号，时间单位毫秒，distance单位像素）

```

ConnectManager.share().writePrinterHeadParameters(int printer_head, int l_pix, int
p_pix, int distance);
// 事件注册和监听 同 readPrinterHeadParameters()

```

33、public void writePrinterHeadParameters(int printer_head, int l_pix, int p_pix, int distance, int delayTime)延时delayTime毫秒设置打印头及打印分辨率及延迟打印距离（打印头分1、2、3、4个序号，时间单位毫秒）

```

ConnectManager.share().writePrinterHeadParameters(int printer_head, int l_pix, int
p_pix, int distance, int delayTime);

```

34、public void readCirculationAndRepeatTime()读取打印机的打印循环次数和重复次数

```

ConnectManager.share().readCirculationAndRepeatTime();
// 事件注册和监听 同 readPrinterHeadParameters()

```

35、public void readCirculationAndRepeatTime(int delayTime) 延时delayTime毫秒读取打印机的打印循环次数和重复次数

```

ConnectManager.share().readCirculationAndRepeatTime(int delayTime);
// 事件注册和监听 同 readPrinterHeadParameters()

```

36、public void writeCirculationAndRepeatTime(int circulation_time, int repeat_time) 设置打印机循环打印次数和重复打印次数

```

ConnectManager.share().writeCirculationAndRepeatTime(int circulation_time, int
repeat_time);

```

37、public void writeCirculationAndRepeatTime(int circulation_time, int repeat_time, int delayTime) 延时delayTime毫秒重复设置打印机循环打印次数和重复打印次数


```
ConnectManager.share().writeCirculationAndRepeatTime(int circulation_time, int repeat_time, int delayTime);
```

38、public void readPrintDirection()读取打印方向

```
ConnectManager.share().readPrintDirection();  
// 事件注册和监听 同 readPrinterHeadParameters()
```

39、public void readPrintDirection(int delayTime) 延时delayTime毫秒读取打印方向

```
ConnectManager.share().readPrintDirection(int delayTime);  
// 事件注册和监听 同 readPrinterHeadParameters()
```

40、public void writePrintDirection(int horizontalDirection, int verticalDirection)设置水平和垂直打印方向

```
/**  
 * 设置打印方向 仅支持 0和1, 其他值将视为无效  
 * horizontalDirection 0、1  
 * verticalDirection 0、1  
 */  
ConnectManager.share().writePrintDirection(int horizontalDirection, int verticalDirection)
```

41、public void writePrintDirection(int horizontalDirection, int verticalDirection, int delayTime) 延时delayTime毫秒设置水平和垂直打印方向

```
/**  
 * 设置打印方向 仅支持 0和1, 其他值将视为无效  
 * horizontalDirection 0、1  
 * verticalDirection 0、1  
 * delayTime 延时时间 单位 毫秒  
 */  
ConnectManager.share().writePrintDirection(int horizontalDirection, int verticalDirection, int delayTime)
```

42、public void readSoftwareInfo()读取打印机软件信息

```
ConnectManager.share().readSoftwareInfo();  
// 事件注册和监听 同 readPrinterHeadParameters()
```

43、**public void readSoftwareInfo(int delayTime)**延时delayTime毫秒读取打印机软件信息

```
ConnectManager.share().readSoftwareInfo(delayTime);  
// 事件注册和监听 同 readPrinterHeadParameters()
```

44、**public void readBattery()** 读取电量信息

```
ConnectManager.share().readBattery();  
// 事件注册和监听 同 readPrinterHeadParameters()
```

45、**public void readBattery(int delayTime)** 延时delayTime毫秒读取电量信息

```
ConnectManager.share().readBattery();  
// 事件注册和监听 同 readPrinterHeadParameters(int delayTime)
```

46、**public static void LogoImage2Data(Context context, LogoImage logoImage, int threshold, boolean transparentToWhiteAuto, OnCreateLogoDataListener onCreateLogoDataListener)**生成默认打印内容数据工具方法，该方法内部执行线程，并以事件的形式返回生成结果

```
// 打印机默认打印内容(logo)对象, 该对象通过LogoImage类的工厂方法createInstance来生成
LogoImage logoImage = LogoImage.createInstance("xxx/xxx.png");
/**
 *
 * @param context 上下文对象
 * @param logoImage logo图片对象
 * @param threshold 二值化阈值
 * @param transparentToWhiteAuto 是否自动将透明背景图转化为白色背景图(通常在明确知道图片不为
透明图片的情况下不建议打开, 检测图片透明会降低运行效率)
 * @param onCreateLogoDataListener logo图片对象处理事件
 */
LogoDataFactory.LogoImage2Data(this, logoImage, 127, false, new
LogoDataFactory.OnCreateLogoDataListener() {
    @Override
    public void onCreateLogoDataStart() {
        //开始处理logo图片
    }

    @Override
    public void onCreateLogoDataComplete(LogoData logoData) {
        //完成图片转化, 并返回LogoData对象
    }

    @Override
    public void onCreateLogoDataError(int code) {
        //发生异常报错
    }
});
```

47、public static LogoData LogoImage2Data(Context context, LogoImage logoImage, int threshold, boolean transparentToWhiteAuto)生成默认打印内容数据工具方法（需用户自己创建线程来执行）

```
// 打印机默认打印内容(logo)对象, 该对象通过LogoImage类的工厂方法createInstance来生成
LogoImage logoImage = LogoImage.createInstance("xxx/xxx.png");
/**
 *
 * @param context 上下文对象
 * @param logoImage logo图片对象
 * @param threshold 二值化阈值
 * @param transparentToWhiteAuto 是否自动将透明背景图转化为白色背景图(通常在明确知道图片不为
透明图片的情况下不建议打开, 检测图片透明会降低运行效率)
 */
LogoData logoData = LogoDataFactory.LogoImage2Data(this, logoImage, 127, false);
```

48、 `public static void bitmap2MultiRowData(Context context, MultiRowImage multiRowImage, int threshold, boolean clearBackground, boolean dithering, boolean compress, boolean flipHorizontally, boolean transparentToWhiteAuto, boolean thumbToSimulation, OnCreateMultiRowDataListener onCreateMultiRowDataListener)` 生成打印数据，该方法内部自己维护线程，并以事件的方式返回执行结果

```
// rowImage 单拼图像对象
RowImage rowImage = RowImage.createInstance("xxx/xxx.png");
String thumbPath = "xxx/xxx.png";
ArrayList<RowImage> rowImages = new ArrayList<>();
rowImages.add(rowImage);
// multiRowImage 多拼图像对象
MultiRowImage multiRowImage = MultiRowImage.createInstance(rowImages, thumbPath);
/**
 *
 * @param context 上下文对象
 * @param multiRowImage 多拼图像对象
 * @param threshold 二值化阈值
 * @param clearBackground 是否移除背景色
 * @param dithering 是否开启抖动算法
 * @param compress 是否启用压缩
 * @param flipHorizontally 是否横向翻转图像
 * @param transparentToWhiteAuto 是否将透明图片转为白色背景图
 * @param thumbToSimulation 缩略图是否转为可预览的打印效果图
 * @param onCreateMultiRowDataListener 图像转化事件
 */
MultiRowDataFactory.bitmap2MultiRowData(this, multiRowImage, 127, false, true, false,
false, false, false, new MultiRowDataFactory.OnCreateMultiRowDataListener() {
    @Override
    public void onCreateMultiRowDataStart() {
        //开始事件
    }

    @Override
    public void onCreateMultiRowDataComplete(MultiRowData multiRowData) {
        //图片处理完成，并在该事件返回MultiRowData对象
    }

    @Override
    public void onCreateMultiRowDataError(int code) {
        //转化过程生成异常
    }
});
```

49、 `public static MultiRowData bitmap2MultiRowData(Context context, MultiRowImage multiRowImage, int threshold, boolean clearBackground, boolean dithering, boolean`

compress, boolean flipHorizontally, boolean transparentToWhiteAuto, boolean thumbToSimulation) 生成打印数据，图片处理过程为耗时任务，需要在使用时创建线程来执行

```
// rowImage 单拼图像对象
RowImage rowImage = RowImage.createInstance("xxx/xxx.png");
String thumbPath = "xxx/xxx.png";
ArrayList<RowImage> rowImages = new ArrayList<>();
rowImages.add(rowImage);
// multiRowImage 多拼图像对象
MultiRowImage multiRowImage = MultiRowImage.createInstance(rowImages,thumbPath);
/**
 *
 * @param context 上下文对象
 * @param multiRowImage 多拼图像对象
 * @param threshold 二值化阈值
 * @param clearBackground 是否移除背景色
 * @param dithering 是否开启抖动算法
 * @param compress 是否启用压缩
 * @param flipHorizontally 是否横向翻转图像
 * @param transparentToWhiteAuto 是否将透明图片转为白色背景图
 * @param thumbToSimulation 缩略图是否转为可预览的打印效果图
 */
MultiRowData multiRowData1 = MultiRowDataFactory.bitmap2MultiRowData(this,
multiRowImage, 127, false, true, false, false, false, false);
```

50、 public static void image2MultiRowImage(Context context, Uri uri, RowLayoutDirection rowLayoutDirection, OnCreateMultiRowImageListener onCreateMultiRowImageListener)用来生成 MultiRowImage

```

/**
 *
 * @param context 上下文对象
 * @param uri 图片的uri
 * @param rowLayoutDirection 裁剪方向
 * @param onCreateMultiRowImageListener 事件
 */
MultiRowImageFactory.image2MultiRowImage(context, uri, rowLayoutDirection, new
MultiRowImageFactory.OnCreateMultiRowImageListener() {
    @Override
    public void onCreateMultiRowImageStart() {

    }

    @Override
    public void onCreateMultiRowImageComplete(MultiRowImage multiRowImage) {
        //生成multiRowImage
    }

    @Override
    public void onCreateMultiRowImageError(int code) {

    }

});

```

51、**public static void image2MultiRowImage(Context context, Uri uri, RowLayoutDirection rowLayoutDirection, int ignoreLastRowIfHeightLess, OnCreateMultiRowImageListener onCreateMultiRowImageListener)**用来生成MultiRowImage

```

/**
 *
 * @param context 上下文
 * @param uri 图片的uri
 * @param rowLayoutDirection 裁剪方向
 * @param ignoreLastRowIfHeightLess 忽略参数, 最后一拼低于该值将直接被忽略
 * @param onCreateMultiRowImageListener 事件
 */
MultiRowImageFactory.image2MultiRowImage(context, uri, rowLayoutDirection,
ignoreLastRowIfHeightLess, new MultiRowImageFactory.OnCreateMultiRowImageListener() {
    @Override
    public void onCreateMultiRowImageStart() {

    }

    @Override
    public void onCreateMultiRowImageComplete(MultiRowImage multiRowImage) {
        //生成multiRowImage
    }

    @Override
    public void onCreateMultiRowImageError(int code) {

    }
});

```

52、public static void image2MultiRowImage(Context context, String imagePath, RowLayoutDirection rowLayoutDirection, OnCreateMultiRowImageListener onCreateMultiRowImageListener)用来生成 MultiRowImage

```

/**
 *
 * @param context 上下文
 * @param imagePath 图片的路径
 * @param rowLayoutDirection 裁剪方向
 * @param onCreateMultiRowImageListener 事件
 */
MultiRowImageFactory.image2MultiRowImage(context, imagePath, rowLayoutDirection, new
MultiRowImageFactory.OnCreateMultiRowImageListener() {
    @Override
    public void onCreateMultiRowImageStart() {
    }

    @Override
    public void onCreateMultiRowImageComplete(MultiRowImage multiRowImage) {
    }

    @Override
    public void onCreateMultiRowImageError(int code) {
    }
});

```

53、public static void image2MultiRowImage(Context context, String imagePath, RowLayoutDirection rowLayoutDirection, int ignoreLastRowIfHeightLess, OnCreateMultiRowImageListener onCreateMultiRowImageListener)用来生成 MultiRowImage


```

/**
 *
 * @param context 上下文
 * @param imagePath 图片的路径
 * @param rowLayoutDirection 裁剪方向
 * @param ignoreLastRowIfHeightLess 忽略参数，最后一拼低于该值将直接被忽略
 * @param onCreateMultiRowImageListener 事件
 */
MultiRowImageFactory.image2MultiRowImage(context, imagePath, rowLayoutDirection,
ignoreLastRowIfHeightLess, new MultiRowImageFactory.OnCreateMultiRowImageListener() {
    @Override
    public void onCreateMultiRowImageStart() {
    }

    @Override
    public void onCreateMultiRowImageComplete(MultiRowImage multiRowImage) {
        progressDialog.dismiss();
        createMultiRowData(multiRowImage);
    }

    @Override
    public void onCreateMultiRowImageError(int code) {
    }
});

```

54、**public static void image2MultiRowImage(Context context, Bitmap bitmap, RowLayoutDirection rowLayoutDirection, OnCreateMultiRowImageListener onCreateMultiRowImageListener)**用来生成 MultiRowImage

```

/**
 *
 * @param context 上下文
 * @param bitmap 图片bitmap对象
 * @param rowLayoutDirection 裁剪方向
 * @param onCreateMultiRowImageListener 事件
 */
MultiRowImageFactory.image2MultiRowImage(context, bitmap, rowLayoutDirection, new
MultiRowImageFactory.OnCreateMultiRowImageListener() {
    @Override
    public void onCreateMultiRowImageStart() {
    }

    @Override
    public void onCreateMultiRowImageComplete(MultiRowImage multiRowImage) {
    }

    @Override
    public void onCreateMultiRowImageError(int code) {
    }
});

```

55、public static void image2MultiRowImage(Context context, Bitmap bitmap, RowLayoutDirection rowLayoutDirection, int ignoreLastRowIfHeightLess, OnCreateMultiRowImageListener onCreateMultiRowImageListener)用来生成 MultiRowImage

```

/**
 *
 * @param context 上下文
 * @param bitmap 图片bitmap对象
 * @param rowLayoutDirection 裁剪方向
 * @param ignoreLastRowIfHeightLess 忽略参数，最后一拼低于该值将直接被忽略
 * @param onCreateMultiRowImageListener 事件
 */
MultiRowImageFactory.image2MultiRowImage(context, bitmap, rowLayoutDirection,
ignoreLastRowIfHeightLess, new MultiRowImageFactory.OnCreateMultiRowImageListener() {
    @Override
    public void onCreateMultiRowImageStart() {
    }

    @Override
    public void onCreateMultiRowImageComplete(MultiRowImage multiRowImage) {
    }

    @Override
    public void onCreateMultiRowImageError(int code) {
    }
});

```

56、public void startPrint() 发送打印指令，和打印机上打印按钮功能相同

```

ConnectManager.share().startPrint();
//注册监听
ConnectManager.share().registerReadPrintStartCommandListener(onReadPrintStartCommandListener);
//注销监听
ConnectManager.share().unregisterReadPrintStartCommandListener(onReadPrintStartCommandListener);
//该指令发送到打印机收到反馈的监听
ConnectManager.OnReadPrintStartCommandListener onReadPrintStartCommandListener = new
ConnectManager.OnReadPrintStartCommandListener() {
    @Override
    public void onReadStartPrintCommand() {
        RBQLog.i("收到打印机反馈的打印指令");
    }
};

```

四、常用类讲解

4.1、com.mx.mxSdk.Device类：打印机设备

该对象通常为打印机设备，这里包括蓝牙打印机设备、wifi设备打印机设备、及直接串口连接的设备，通常无需自己创建，只需要调用相应的扫描或者get方法即可获得该对象。如：
discoverBluetoothDevice（扫描蓝牙设备）、discoverWifiDevice（扫描wifi设备）和
getSerialDevices（获取串口设备），其示例见上边api列表中discoverBluetoothDevice方法的使用示例。

4.2、com.mx.mxSdk.DistNetDevice类：配网设备

该对象为配网设备，既调用discoverDistNetDevice方法搜索配网设备，由它的事件
OnDistNetDeviceDiscoverListener返回的对象，该对象也无需自己创建。其示例见上边api列表中
discoverDistNetDevice方法的使用示例。

4.3、com.mx.mxSdk.LogoImage类：打印机默认打印内容（logo）

```
public class LogoImage implements Parcelable {  
  
    private final String imagePath;  
}  
// 该对象仅提供一个工厂方法createInstance来创建该对象，需要传入logo图片的存储路径  
LogoImage logoImage = LogoImage.createInstance("xxx/xxx.png");
```

4.4、com.mx.mxSdk.LogoData类 打印默认图片生成的数据对象，该对象通常无需自己创建，调用LogoDataFactory工厂类的LogoImage2Data方法生成

```
public class LogoData implements Parcelable {  
    /** 数据长度 */  
    private final int dataLength;  
    /** 数据存储的路径 */  
    private final String dataPath;  
    /** 生成的预览图存储的路径 */  
    private final String imagePath;  
}
```

4.5、com.mx.mxSdk.LogoDataFactory类 用于将LogoImage转为打印机默认打印数据 LogoData

```

String logoImagePath = "xxx/xxx.png";
LogoImage logoImage = LogoImage.createInstance(logoImagePath);

/** 调用该方法，则可直接返回logoData 但是需要自己在外部创建线程来维持该耗时任务 */
LogoData logoData = LogoDataFactory.LogoImage2Data(this, logoImage, 127, false);
/** 调用该方法，传入一个事件，结果以事件的形式返回，无需外边单独维持线程 */
LogoDataFactory.LogoImage2Data(this, logoImage, 127, false, new
LogoDataFactory.OnCreateLogoDataListener() {
    @Override
    public void onCreateLogoDataStart() {
        //开始处理logo图片
    }

    @Override
    public void onCreateLogoDataComplete(LogoData logoData) {
        //完成logo图片处理，并返回 logoData 对象
    }

    @Override
    public void onCreateLogoDataError(int code) {
        //处理异常
    }
});

//发送打印数据
ConnectManager.share().setWithSendLogoPacket(logoData);

```

4.6、com.mx.mxSdk.MultiRowImage类：多拼图或者单张大图裁切形成的连续图

```

public class MultiRowImage implements Parcelable {

    private final ArrayList<RowImage> rowImages;
    /** 缩略图地址*/
    private final String thumbPath;
    /** 多张图的排布方向 RowLayoutDirectionVertical 从上到下排列
RowLayoutDirectionHorizontal从左到右排列    */
    private final RowLayoutDirection rowLayoutDirection;
    /** 图片是否为连续图（既由一张大图裁切而来的连续图片） */
    private final boolean isContiguousCroppedImages;
}

// 创建MultiRowImage对象示例
MultiRowImage multiRowImage = MultiRowImage.createInstance(rowImages, thumbPath);
MultiRowImage multiRowImage1 = MultiRowImage.createInstance(rowImages, thumbPath,
RowLayoutDirectionHorizontal, true);

```

4.7、com.mx.mxSdk.MultiRowData类 多拼图或者单张大图裁切形成的连续图生成的打印数据

```
public class MultiRowData implements Parcelable {
    /** 多拼图或者单张大图裁切形成的连续图生成的打印数据 */
    private final ArrayList<RowData> rowDataArr;
    /** 多拼图或者单张大图裁切形成的连续图生成的预览图的存储地址 */
    private final ArrayList<String> imagePaths;
    /** 缩略图生成的地址 thumbPath有可能为null,
     * 是否将缩略图生成预览图需要看MultiRowDataFactory的bitmap2MultiRowData方法传入的
     thumbToSimulation的值
     * thumbToSimulation为true则生成预览图, thumbToSimulation为false则不将缩略图生成预览图
     */
    private final String thumbPath;
    /** 是否压缩打印数据 有损压缩 */
    private final boolean compress;
    /** 多图的排布方向, 该值和MultiRowImage的rowLayoutDirection保持一致 */
    private final RowLayoutDirection rowLayoutDirection;
}
```

4.8、com.mx.mxSdk.MultiRowDataFactory 类 用于将多拼打印图MultiRowImage生成打印数据的工具类

```

String imagePath = "xxx/xxx.png";
String thumbPath = "xxx/xxx.png";
RowImage rowImage = RowImage.createInstance(imagePath);
ArrayList<RowImage> rowImages = new ArrayList<RowImage>();
rowImages.add(rowImage);
MultiRowImage multiRowImage = MultiRowImage.createInstance(rowImages,thumbPath);
// 该方法将多拼打印图转化为打印数据，直接返回multiRowData的值，该耗时事件需要用户自己维持线程
MultiRowData multiRowData = MultiRowDataFactory.bitmap2MultiRowData(this,
multiRowImage, 127, true, false, false, false,false,false);
//该方法将多拼打印图转化为打印数据，该方法内部自己维护线程，无需用户单独外部创建线程，通过事件的形式返回结果
MultiRowDataFactory.bitmap2MultiRowData(this, multiRowImage, 127, true, false, false,
false
    , false, false, new MultiRowDataFactory.OnCreateMultiRowDataListener() {
        @Override
        public void onCreateMultiRowDataStart() {

        }

        @Override
        public void onCreateMultiRowDataComplete(MultiRowData multiRowData) {

        }

        @Override
        public void onCreateMultiRowDataError(int code) {

        }
    });

//发送打印数据
ConnectManager.share().setWithSendMultiRowDataPacket(multiRowData);

```

五、关于打印开始和完成事件的监听

```

//事件注册
ConnectManager.share().registerPrintListener(onPrintListener);
//事件注销
ConnectManager.share().unregisterPrintListener(onPrintListener);
//事件监听
ConnectManager.OnPrintListener onPrintListener = new ConnectManager.OnPrintListener() {
    @Override
    public void onPrintStart(int beginIndex, int endIndex, int currentIndex) {
        RBQLog.i("打印开始事件 beginIndex:"+beginIndex+"; endIndex:"+endIndex+";
currentIndex:"+currentIndex);
    }

    @Override
    public void onPrintComplete(int beginIndex, int endIndex, int currentIndex,
String cartridgeId) {
        RBQLog.i("打印结束事件 beginIndex:"+beginIndex+"; endIndex:"+endIndex+";
currentIndex:"+currentIndex+"; 墨盒cartridgeId:"+cartridgeId);
    }
};

```

六、TransportProtocol（传输）协议类型简介

以下常亮分别代表了协议的类型，数据结构为 $fn + \sim fn + data + crc$ 。实际长度为 $1 + 1 + data.length + 2$ ，

蓝牙类型的打印机：在安卓中使用的是spp协议，推荐使用STX_A，在iOS中使用尽可能使用STX_E (具体开发还需和我们沟通，不同打印机所支持的协议种类存存在差异)

wifi类型的打印机：推荐使用 STX_A 协议进行传输


```

public final class TransportProtocol {
    // 每包有效数据长度为128byte（打印机未支持）
    public static final byte SOH = 0x18;
    // 每包有效数据长度为512byte
    public static final byte STX = 0x19;
    // 每包有效数据长度为 1024byte
    public static final byte STX_A = 0x1A;
    // 每包有效数据长度为 2KB
    public static final byte STX_B = 0x1B;
    // 未使用和验证
    public static final byte STX_C = 0x1C;
    // 打印机mcu暂时未支持
    public static final byte STX_D = 0x1D;
    // 打印机mcu支持
    public static final byte STX_E = 0x1E;
}

```

七、工厂类中的错误码含义

```

public class FactoryErrorCodes {
    /**上下文对象为null*/
    public static final int Context_NULL_ERROR = 1 << 0;
    /**图片路径为null*/
    public static final int IMAGE_PATH_NULL_ERROR = 1 << 1;
    /**图片为null*/
    public static final int IMAGE_NULL_ERROR = 1 << 2;
    /**RowImage为null*/
    public static final int ROW_IMAGE_NULL_ERROR = 1 << 3;
    /**MultiRowImage为null*/
    public static final int MULTI_ROW_IMAGE_NULL_ERROR = 1 << 4;
    /**MultiRowImage创建失败null*/
    public static final int MULTI_ROW_IMAGE_CREATION_FAILED = 1 << 5;
    /**文件没有找到*/
    public static final int FILE_NOT_FOUND = 1 << 6;
    /**io异常*/
    public static final int IOException_ERROR = 1 << 7; // 16
}

```

八、其它常用监听器

本节补充 SDK 中未在 API 章节内联说明的监听器及其使用方式。

1、OnDeviceBluetoothStateListener 蓝牙状态监听

蓝牙打开/关闭过程事件。

```
//事件注册
ConnectManager.share().registerDeviceBluetoothStateListener(onDeviceBluetoothStateListener);
//事件注销
ConnectManager.share().unregisterDeviceBluetoothStateListener(onDeviceBluetoothStateListener);

//事件示例
ConnectManager.OnDeviceBluetoothStateListener onDeviceBluetoothStateListener = new
ConnectManager.OnDeviceBluetoothStateListener() {
    @Override
    public void onDeviceBlueToothOpening() {
        //蓝牙正在打开
    }

    @Override
    public void onDeviceBlueToothOpened() {
        //蓝牙已打开
    }

    @Override
    public void onDeviceBlueToothClosing() {
        //蓝牙正在关闭
    }

    @Override
    public void onDeviceBlueToothClosed() {
        //蓝牙已关闭
    }
};
```

2、OnDeviceBondListener 设备配对监听

配对、A2DP、Socket 等事件。

```
//事件注册
ConnectManager.share().registerDeviceBondListener(onDeviceBondListener);
//事件注销
ConnectManager.share().unregisterDeviceBondListener(onDeviceBondListener);

//事件示例
ConnectManager.OnDeviceBondListener onDeviceBondListener = new
ConnectManager.OnDeviceBondListener() {
    @Override
    public void onDeviceBonding(Device device) {
        //正在与设备配对
    }

    @Override
    public void onDeviceBonded(Device device) {
        //配对成功
    }

    @Override
    public void onDeviceDisBond(Device device) {
        //取消配对
    }
};
```

3、OnCommandWriteListener 指令写入监听

SDK 内部指令写入成功/失败事件。

```

//事件注册
ConnectManager.share().registerCommandWriteListener(onCommandWriteListener);
//事件注销
ConnectManager.share().unregisterCommandWriteListener(onCommandWriteListener);

//事件示例
ConnectManager.OnCommandWriteListener onCommandWriteListener = new
ConnectManager.OnCommandWriteListener() {
    @Override
    public void onCommandWriteSuccess(Device device, Command command, Object object) {
        //指令写入成功
    }

    @Override
    public void onCommandWriteError(Device device, Command command, String errorMsg) {
        //指令写入失败
    }
};

```

4、OnDataWriteListener 数据写入监听

数据块写入成功/失败事件。

```

//事件注册
ConnectManager.share().registerDataWriteListener(onDataWriteListener);
//事件注销
ConnectManager.share().unregisterDataWriteListener(onDataWriteListener);

//事件示例
ConnectManager.OnDataWriteListener onDataWriteListener = new
ConnectManager.OnDataWriteListener() {
    @Override
    public void onDataWriteSuccess(Device device, DataObj dataObj, Object object) {
        //数据写入成功
    }

    @Override
    public void onDataWriteError(Device device, DataObj dataObj, String errorMsg) {
        //数据写入失败
    }
};

```

5、OnDataSynchronizeListener 数据同步监听

多拼数据发送过程中同步状态事件。

```
//事件注册
ConnectManager.share().registerDataSynchronizeListener(onDataSynchronizeListener);
//事件注销
ConnectManager.share().unregisterDataSynchronizeListener(onDataSynchronizeListener);

//事件示例
ConnectManager.OnDataSynchronizeListener onDataSynchronizeListener = new
ConnectManager.OnDataSynchronizeListener() {
    @Override
    public void onDataSynchronizeStart(Device device) {
        //数据同步开始
    }

    @Override
    public void onDataSynchronizeComplete(Device device) {
        //数据同步完成
    }

    @Override
    public void onDataSynchronizeTimeout(Device device, int pendingCount) {
        //数据同步超时, pendingCount 为未响应指令数量
    }

    @Override
    public void onDataSynchronizeInterrupted(Device device) {
        //数据同步被中断（如设备断连）
    }
};
```

6、OnReadGeneralCommandListener 通用指令回读监听

读取打印机自定义通用指令响应。

```

//事件注册
ConnectManager.share().registerReadGeneralCommandListener(onReadGeneralCommandListener)
;
//事件注销
ConnectManager.share().unregisterReadGeneralCommandListener(onReadGeneralCommandListene
r);

//事件示例
ConnectManager.OnReadGeneralCommandListener onReadGeneralCommandListener = new
ConnectManager.OnReadGeneralCommandListener() {
    @Override
    public void onReadGeneralCommand(int opcode, String json) {
        //收到通用指令响应, opcode 为操作码, json 为响应内容
    }
};

```

7、OnReadJsonListener JSON 数据回读监听

读取打印机主动上报的 JSON 数据。

```

//事件注册
ConnectManager.share().registerReadJsonListener(onReadJsonListener);
//事件注销
ConnectManager.share().unregisterReadJsonListener(onReadJsonListener);

//事件示例
ConnectManager.OnReadJsonListener onReadJsonListener = new
ConnectManager.OnReadJsonListener() {
    @Override
    public void onReadJson(Device device, String json) {
        //收到打印机上报的 JSON 数据
    }
};

```